

**LAPORAN PRAKTIKUM II  
SISTEM OPERASI (PROCESS)**



**Dosen Pengampu : Triawan Adi Cahyanto, M.K**

**Di susun Oleh:**

**Muhammad maulana wiramatajaya 2410651006**

**PROGRAM STUDY TEKNIK INFORMATIKA**

**FAKULTAS TEKNIK**

**UNIVERSITAS MUHAMMADIYAH JEMBER**

**2024**

# 1. TUJUAN

## 1.1. Process Concept dan Process Operation

1. Memahami konsep proses dan siklus hidupnya
2. Mampu membuat dan mengelola proses menggunakan system call
3. Memahami hubungan parent-child process
4. Menganalisis Process ID (PID) dan Process Control Block.

## 1.2. Interprocess Communication (IPC)

1. Memahami konsep komunikasi antar proses
2. Mengimplementasikan Shared Memory IPC
3. Mengimplementasikan Message Passing dengan Pipes
4. Membandingkan performa kedua metode

# 2. DASAR TEORI

## 2.1. KONSEP PROCESS

Secara informal; proses adalah program dalam eksekusi. Suatu proses adalah lebih dari kode program, dimana kadang kala dikenal sebagai bagian tulisan. Proses juga termasuk aktivitas yang sedang terjadi, sebagaimana digambarkan oleh nilai pada program counter dan isi dari daftar prosesor/ processor's register. Suatu proses umumnya juga termasuk process stack, yang

berisikan data temporer (seperti parameter metoda, address yang kembali, dan variabel lokal) dan sebuah data section, yang berisikan variabel global. Kami tekankan bahwa program itu sendiri bukanlah sebuah proses; suatu program adalah satu entitas pasif; seperti isi dari sebuah berkas yang disimpan didalam disket, sebagaimana sebuah proses dalam suatu entitas aktif, dengan sebuah program counter yang mengkhususkan pada instruksi selanjutnya untuk dijalankan dan seperangkat sumber daya/ resource yang berkenaan dengannya.

Walau dua proses dapat dihubungkan dengan program yang sama, program tersebut dianggap dua urutan eksekusi yang berbeda. Sebagai contoh, beberapa pengguna dapat menjalankan copy yang berbeda pada mail program, atau pengguna yang sama dapat meminta banyak copy dari program editor. Tiap-tiap proses ini adakah proses yang berbeda dan walau bagian tulisan-text Adalah sama,

data section bervariasi. Juga adalah umum untuk memiliki proses yang menghasilkan banyak proses begitu ia bekerja.

### 3. PERCOBAAN

#### 3.1. SISTEM MANAGEMENT PROCESS

Buat File **basic\_process.c** lalu masukan kode di bawah

```
pertemuan_2 > praktikum > C basic_process.c
1  #include <stdio.h>
2  #include <unistd.h>
3  #include <sys/types.h>
4  int main() {
5  pid_t pid;
6  printf("Program dimulai\n");
7  printf("PID proses ini: %d\n", getpid());
8  printf("PID parent: %d\n", getppid());
9  return 0;
10 }
```

Untuk mengCompile file proses\_manajemen Tulis kode **gcc -o main basic\_process.c** Jika tidak ada pesan error, langsung running dengan cara **./main**. Maka hasilnya seperti gambar dibawah.

```
wiramatajaya@LAPTOP-7B38G7J9:~/pertemuan_2/praktikum$ gcc -o main basic_process.c
wiramatajaya@LAPTOP-7B38G7J9:~/pertemuan_2/praktikum$ ./main
Program dimulai
PID proses ini: 1873
PID parent: 1510
wiramatajaya@LAPTOP-7B38G7J9:~/pertemuan_2/praktikum$
```

### 3.2. Parent dan Child Process

Buat File `fork_demo.c` lalu masukan kode di bawah.

```
pertemuan_2 > praktikum > C fork_demo.c
1  #include <stdio.h>
2  #include <unistd.h>
3  #include <sys/types.h>
4  int main() {
5      pid_t pid;
6      printf("Sebelum fork()\n");
7      printf("PID: %d\n\n", getpid());
8      pid = fork();
9      if (pid < 0) {
10         fprintf(stderr, "Fork gagal!\n");
11         return 1;
12     }
13     else if (pid == 0) {
14         // Child process
15         printf("CHILD PROCESS\n");
16         printf("PID saya: %d\n", getpid());
17         printf("PID parent saya: %d\n", getppid());
18     }
19     else {
20         // Parent process
21         printf("PARENT PROCESS\n");
22         printf("PID saya: %d\n", getpid());
23         printf("PID child saya: %d\n", pid);
24     }
25     printf("Proses %d selesai\n\n", getpid());
26     return 0;
27 }
```

Untuk mengCompile file proses\_manajemen Tulis kode `gcc -o fork_demo fork_demo.c` Jika tidak ada pesan error, langsung running dengan cara `./fork_demo`. Maka hasilnya seperti gambar dibawah.

```
wiramatajaya@LAPTOP-7B38G7J9:~/pertemuan_2/praktikum$ gcc -o fork_demo fork_demo.c
wiramatajaya@LAPTOP-7B38G7J9:~/pertemuan_2/praktikum$ ./fork_demo
Sebelum fork()
PID: 1892

PARENT PROCESS
PID saya: 1892
PID child saya: 1893
Proses 1892 selesai

CHILD PROCESS
PID saya: 1893
PID parent saya: 1892
Proses 1893 selesai

wiramatajaya@LAPTOP-7B38G7J9:~/pertemuan_2/praktikum$
```

### 3.3 Process Termination dengan wait()

Buat File `process_wait.c` lalu masukan kode di bawah.

```
pertemuan_2 > praktikum > C process_wait.c
1  #include <stdio.h>
2  #include <unistd.h>
3  #include <sys/types.h>
4  int main() {
5  pid_t pid;
6  printf("Sebelum fork()\n");
7  printf("PID: %d\n\n", getpid());
8  pid = fork();
9  if (pid < 0) {
10 fprintf(stderr, "Fork gagal!\n");
11 return 1;
12 }
13 else if (pid == 0) {
14 // Child process
15 printf("CHILD PROCESS\n");
16 printf("PID saya: %d\n", getpid());
17 printf("PID parent saya: %d\n", getppid());
18 }
19 else {
20 // Parent process
21 printf("PARENT PROCESS\n");
22 printf("PID saya: %d\n", getpid());
23 printf("PID child saya: %d\n", pid);
24 }
25 printf("Proses %d selesai\n\n", getpid());
26 return 0;
27 }
28
29
```

Untuk mengcompile file proses\_manajemen Tulis kode `gcc -o fork_demo fork_demo.c` Jika tidak ada pesan error, langsung running dengan cara `./fork_demo`. Maka hasilnya seperti gambar dibawah.

### 3.3. Interprocess Communication (IPC)

Buat File `process_wait.c` lalu masukan kode di bawah.

```
pertemuan_2 > praktikum > C pipe_demo.c
1  #include <stdio.h>
2  #include <unistd.h>
3  #include <string.h>
4  #include <sys/wait.h>
5  int main() {
6  int pipefd[2]; // pipefd[0] = read, pipefd[1] = write
7  pid_t pid;
8  char write_msg[] = "Hello dari Parent!";
9  char read_msg[100];
10 // Buat pipe
11 if (pipe(pipefd) == -1) {
12 perror("Pipe gagal");
13 return 1;
14 }
15 pid = fork();
16 if (pid == 0) {
17 // Child process - membaca dari pipe
18 close(pipefd[1]); // Tutup write end
19 read(pipefd[0], read_msg, sizeof(read_msg));
20 printf("Child menerima: %s\n", read_msg);
21 close(pipefd[0]);
22 }
23 else {
24 // Parent process - menulis ke pipe
25 close(pipefd[0]); // Tutup read end
26 printf("Parent mengirim: %s\n", write_msg);
27 write(pipefd[1], write_msg, strlen(write_msg) + 1);
28 close(pipefd[1]);
29 wait(NULL);
30 }
31 return 0;
32 }
```

Untuk mengCompile file proses\_manajemen Tulis kode `gcc -o fork_demo fork_demo.c` Jika tidak ada pesan error, langsung running dengan cara `./fork_demo`. Maka hasilnya seperti gambar dibawah.

```
PROBLEMS  OUTPUT  TERMINAL  PORTS
▼ TERMINAL

wiramatajaya@LAPTOP-7B38G7J9:~/pertemuan_2/praktikum$ gcc -o pipe_demo pipe_demo.c
pipe_demo.c: In function 'main':
pipe_demo.c:28:1: warning: implicit declaration of function 'wait' [-Wimplicit-function-declaration]
   28 | wait(NULL);
      | ^~~~~
wiramatajaya@LAPTOP-7B38G7J9:~/pertemuan_2/praktikum$ gcc -o pipe_demo pipe_demo.c
pipe_demo.c: In function 'main':
pipe_demo.c:28:1: warning: implicit declaration of function 'wait' [-Wimplicit-function-declaration]
   28 | wait(NULL);
      | ^~~~~
wiramatajaya@LAPTOP-7B38G7J9:~/pertemuan_2/praktikum$ gcc -o pipe_demo pipe_demo.c
wiramatajaya@LAPTOP-7B38G7J9:~/pertemuan_2/praktikum$ nano pipe_demo.c
wiramatajaya@LAPTOP-7B38G7J9:~/pertemuan_2/praktikum$ gcc -o pipe_demo pipe_demo.c
wiramatajaya@LAPTOP-7B38G7J9:~/pertemuan_2/praktikum$ ./pipe_demo
Parent mengirim: Hello dari Parent!
Child menerima: Hello dari Parent!
wiramatajaya@LAPTOP-7B38G7J9:~/pertemuan_2/praktikum$
```

### 3.4. Named Pipes (FIFO) - Two Way Communication

Buat File **process\_wait.c** lalu masukan kode di bawah

```
pertemuan_2 > praktikum > C fifo_writer.c
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <fcntl.h>
5  #include <sys/stat.h>
6  #include <unistd.h>
7  #define FIFO_FILE "/tmp/my_fifo"
8  int main() {
9  int fd;
10 char buffer[100];
11 // Buat FIFO jika belum ada
12 mkfifo(FIFO_FILE, 0666);
13 printf("Writer: Menunggu reader...\n");
14 fd = open(FIFO_FILE, O_WRONLY);
15 printf("Masukkan pesan: ");
16 fgets(buffer, sizeof(buffer), stdin);
17 write(fd, buffer, strlen(buffer) + 1);
18 printf("Writer: Pesan terkirim\n");
19 close(fd);
20 return 0;
21 }
```

Untuk mengCompile file proses\_manajemen Tulis kode **gcc -o fifo\_writer fifo\_demo.c** Jika tidak ada pesan error, langsung running dengan cara **./fifo\_writer**. Maka hasilnya seperti gambar dibawah.

```
wiramatajaya@LAPTOP-7B38G7J9:~/pertemuan_2/praktikum$ gcc -o fifo_writer fifo_writer.c
wiramatajaya@LAPTOP-7B38G7J9:~/pertemuan_2/praktikum$ ./fifo_writer
Writer: Menunggu reader...
Masukkan pesan: wiramatajaya
Writer: Pesan terkirim
wiramatajaya@LAPTOP-7B38G7J9:~/pertemuan_2/praktikum$
```

```
wiramatajaya@LAPTOP-7B38G7J9:~/pertemuan_2/praktikum$ gcc -o fifo_reader fifo_reader.c
wiramatajaya@LAPTOP-7B38G7J9:~/pertemuan_2/praktikum$ ./fifo_reader
Reader: Membuka FIFO...
Reader: Menerima pesan: wiramatajaya
wiramatajaya@LAPTOP-7B38G7J9:~/pertemuan_2/praktikum$
```

## 4. KESIMPULAN

Berdasarkan hasil praktikum yang telah dilakukan, dapat disimpulkan bahwa:

1. **Konsep Proses (Process Concept)** merupakan dasar dari pengelolaan eksekusi program dalam sistem operasi. Proses adalah program yang sedang dieksekusi, memiliki *state*, *program counter*, *register*, serta *memory space* tersendiri. Melalui praktikum ini, dipahami bahwa sistem operasi berperan penting dalam membuat, menjadwalkan, dan mengatur proses agar dapat berjalan secara efisien dan tidak saling mengganggu.
2. **Operasi pada Proses (Process Operations)** seperti *process creation*, *termination*, dan *scheduling* menjadi mekanisme utama dalam manajemen proses. Praktikum menunjukkan bagaimana proses induk (*parent process*) dapat membuat proses anak (*child process*) menggunakan sistem call seperti *fork()*, serta bagaimana sistem operasi menangani komunikasi dan penghapusan proses. Selain itu, pemahaman tentang *process state transition* (*ready*, *running*, *waiting*, *terminated*) menjadi penting untuk memahami siklus hidup proses.
3. **Komunikasi Antarproses (Interprocess Communication / IPC)** memungkinkan proses yang berbeda untuk saling bertukar data dan berkoordinasi. Melalui mekanisme seperti *pipe*, *message queue*, *shared memory*, dan *semaphore*, sistem operasi menyediakan cara agar proses dapat bekerja secara paralel dan sinkron tanpa konflik data. Praktikum ini memperlihatkan pentingnya sinkronisasi agar tidak terjadi *race condition* atau *deadlock* antarproses.
4. Secara keseluruhan, ketiga praktikum ini memperkuat pemahaman bahwa **manajemen proses dan komunikasi antarproses merupakan inti dari kinerja sistem operasi modern**, karena hampir semua aktivitas sistem bergantung pada efisiensi dan sinkronisasi proses-proses yang berjalan bersamaan.